ASSIGNMENT 4 - CO4

TASK 2 - FROM WORKLOAD TO FILE-ALLOCATION METHOD

Operating Systems (CSA04) | 8 Marks | Bloom's Level: K5 - Evaluate

1. Workload and Design Context

The question describes a network video-surveillance recorder storing footage from 32 cameras. Files grow continuously while recording, are read mainly from start to end, are sometimes searched by time (random access), and are deleted after 30 days. The chosen allocation method must support sustained appends, sequential playback, occasional direct access to a timestamp and practical retention deletion.

The question does not provide camera bit rate, resolution, clip duration, viewer count or a required block size for the free-space calculation. Therefore, exact 30-day storage capacity and aggregate throughput cannot be derived. The calculations below use the stated 4 GB file, 4 KB blocks, 4-byte block addresses and 4 TB disk.

2. Task 2.1 - Operations and Access Patterns

Operation	Frequency / timing	Access pattern and implication
Record/write footage	Continuous and high frequency; 32 concurrent camera streams.	Sequential append; needs sustained throughput and low allocation overhead.
Playback/read footage	Main read workload, mainly from beginning to end.	Primarily sequential; contiguous and linked layouts stream naturally.
Search by timestamp	Occasional compared with continuous recording.	Direct/random access to the block for a time location; requires an index or fast block mapping.
Retention deletion	Periodic removal of footage older than 30 days.	Metadata lookup plus block deallocation; time-window files simplify expiry.
Create/close recording files	At stream start/stop or recording-window rollover.	Directory and metadata operations; timestamp index maps ranges to files.

A practical design can organise recordings by camera ID and bounded time window, with a timestamp-to-file/block index. Hourly or daily clips are examples only; exact segment duration is a design choice because the question does not specify it. Bounded files make 30-day deletion easier than retaining one indefinitely growing file per camera.

3. Task 2.2 - Allocation-Method Comparison

Criterion	Contiguous	Linked (including FAT)	Indexed
Sequential access	Excellent; adjacent blocks can be streamed.	Good; follow next pointer, with pointer overhead.	Good; read data blocks in index order.
Direct/time access	O(1) from start block and offset.	Basic linked: O(k) chain traversal. FAT in RAM: O(k) table lookups.	Direct pointer lookup when index is available; multi-level requires a few index reads.
File growth	Difficult if adjacent free blocks are unavailable; relocation or extents needed.	Easy; append a block and update the previous tail pointer.	Easy; allocate a free block and add its address to the index.
Fragmentation	External fragmentation; a long contiguous run may not exist.	No external fragmentation of data blocks.	No external fragmentation of data blocks; index blocks use metadata space.
Reliability	Simple mapping, but extent metadata loss can make file inaccessible.	Pointer damage can break the chain; FAT damage can affect many files.	Index damage affects mappings for that file; metadata protection and backups are still needed.

Indexed allocation is the best fit for the stated mix: files grow continuously, playback is mostly sequential, and time searches sometimes require direct access. Contiguous allocation is fast but growth is fragile; linked allocation grows easily but random access is slow. Indexed allocation avoids the need for one long free run

while exposing block addresses through the index.

4. Task 2.3 - Disk-Read and Inode Calculations

4.1 Number of blocks in a 4 GB file

Use binary units for the OS calculations: 1 GB = 2^30 bytes, 1 KB = 2^10 bytes, so 4 KB = 2^12 bytes.

4 GB / 4 KB = (4 x 2^30) / 2^12 = 2^20 = 1,048,576 data blocks.

4.2 Reads to reach the last data block

Method	Read-count assumption	Reads
Contiguous	File extent metadata is already available; calculate the last address and read the target block.	1 data-block read
Basic linked	Read each block in sequence to obtain the next pointer, including the final target block.	1,048,576 block reads
Indexed, two levels	4 KB index holds 4,096/4 = 1,024 pointers; the file needs a root index and 1,024 leaf index blocks.	3 cold reads: root index + leaf index + data; 2 if root is cached

A single 4 KB index block cannot hold 1,048,576 pointers. With 1,024 four-byte pointers per block, the root points to 1,024 second-level index blocks, each of which points to 1,024 data blocks. Thus a two-level index is sufficient. The disk-read count assumes the inode/file metadata is available and index blocks are not cached.

If the linked structure uses a FAT held in RAM, reaching block k takes O(k) in-memory FAT lookups and then one read of the target data block. For the last block here that is 1,048,575 next-entry lookups plus one data-block read. RAM lookups are not disk reads, but traversal is still linear.

4.3 Maximum file size from the given inode

Each indirect block holds 4,096 / 4 = 1,024 block addresses.

Pointer type	Addressable data blocks
12 direct pointers	12
Single indirect	1,024
Double indirect	1,024^2 = 1,048,576
Triple indirect	1,024^3 = 1,073,741,824
Total	1,074,791,436 blocks

Maximum = 1,074,791,436 x 4,096 = 4,402,345,721,856 bytes, approximately 4.00391 TiB.

This is the pointer-addressing limit under the specified block and pointer sizes. A real file system may impose additional implementation and volume limits.

5. Task 2.4 - Free-Space Management

Assume 4 TB is represented as 4 TiB = 2^42 bytes and the allocation block is 4 KB = 2^12 bytes.

Blocks = 2^42 / 2^12 = 2^30 = 1,073,741,824.
Bitmap = 2^30 bits / 8 = 2^27 bytes = 134,217,728 bytes = 128 MiB.

Method	Operation	Strength	Limitation
Bit vector	One bit marks each block free/allocated.	Compact; bitmap scan can find free blocks and runs.	Finding a long run can require O(B) scan.
Linked free list	Each free block points to the next; allocate from head.	Simple; allocate one block at head in O(1).	Finding a large contiguous run is O(B); pointer traversal may need I/O.
Grouping	A free block stores several free-block addresses; last entry points to next group.	Allocates batches with fewer list traversals.	Must maintain groups and advance when group is empty.
Counting	Store start block plus count of consecutive free blocks.	Compact when free space is in runs.	Less effective when free space is highly fragmented.

If decimal drive units are intended (4 TB = 4,000,000,000,000 bytes), the bitmap is 122,070,313 bytes, about 116.4 MiB. The main answer uses the binary OS-textbook convention.

6. Complete the Code - First-Fit Contiguous Allocation

The supplied program leaves TODO 1 in alloc_contiguous(). The completed function scans from block 0, counts consecutive free blocks, marks the first run that reaches the requested size and returns its start. It returns -1 if no run is long enough. The rest of the supplied program is preserved in file_allocation.c.

```
int run = 0;

for (int b = 0; b < BLOCKS; b++) {
    if (disk[b] == '.')
        run++;
    else
        run = 0;

    if (run == size) {
        int start = b - size + 1;

        for (int i = start; i <= b; i++)
            disk[i] = file;

        return start;
    }
}

return -1;
```

Compile using the assignment's required command:

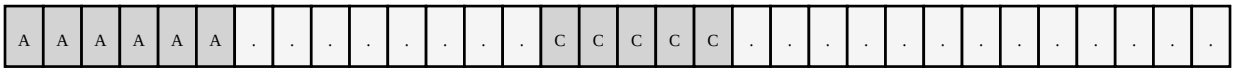
```
gcc -std=c11 -Wall -Wextra file_allocation.c -o file_allocation
```

The program was compiled and executed. Its output matches the expected output in the question paper:

```
AAAAAA........CCCCC.............
E (10 blocks) contiguous: start 19
E (10 blocks) indexed: index block 6, data blocks 7 8 9 10 11 12 13 29 30 31
AAAAAAeEEEEEEECCCCCEEEEEEEEEEEE
```

7. Clarification of the Fragmentation Statement

Free-space map after releasing B and D



Free runs: blocks 6-13 (8) and 19-31 (13); total = 21 free blocks.

The question asks why E (10 blocks) cannot be stored contiguously even though 21 blocks are free. However, the supplied state and expected output contradict that wording. After B and D are released, free runs are blocks 6-13 (8 blocks) and 19-31 (13 blocks). The second run is long enough for a 10-block file.

Therefore, first-fit correctly allocates E contiguously at block 19. The subsequent indexed allocation uses the remaining free blocks: block 6 for the index, and blocks 7-13 plus 29-31 for the ten data blocks. The expected final map is AAAAAAeEEEEEEECCCCCEEEEEEEEEEEE. It would be inaccurate to state that the initial contiguous allocation fails; both the source code and expected output show success. Indexed allocation demonstrates how the remaining scattered blocks can also be used.

8. Alternative Concept - FAT Held in Memory

A FAT stores next-block pointers in a table indexed by block number. It avoids reading every video block just to discover the next block, but still follows the chain entry by entry. Access to block k takes O(k) FAT lookups in RAM plus one read of the target data block. Indexed allocation uses O(1) pointer access when an index block is available, or a small number of index reads for multilevel indexing.

For a 4 TB disk with 4 KB blocks there are 2^30 blocks. With one 4-byte FAT entry per block:

2^30 x 4 bytes = 2^32 bytes = 4 GiB of FAT memory.

This is a substantial memory cost for a recorder that also needs RAM for video buffering. FAT centralisation creates a broad failure risk: corruption can make many file chains inaccessible or misdirect allocation. Mirrored/backup FAT copies and consistency checks reduce this risk. Indexed allocation localises mapping metadata to a file's index structure; damage can still affect that file, but does not normally corrupt every file's chain.

9. Recommendation and Execution

Use indexed allocation with multi-level indexing (or an extent-based implementation with an index of time ranges) for the 32-camera recorder. It supports continuous growth without a long contiguous free run and allows direct location of blocks by timestamp, while sequential playback remains efficient. Organise footage by camera and bounded time window so 30-day expiry can remove complete segments. Contiguous allocation is fragile for growing files; basic linked allocation makes time seeks slow; FAT requires O(k) RAM lookups and 4 GiB for the assumed 4 TB / 4 KB geometry.

To execute the supplied program, open Ubuntu/WSL in the Task02 folder, run chmod +x run_task2.sh, and then run ./run_task2.sh. Confirm that the script prints PASS and capture the terminal output. Keep the report, README, C source, expected output, actual output, calculations and environment information together.

Source alignment: Assignment 4 - CO4, Task 2, page 34; Complete-the-Code and expected output, page 35; FAT alternative and evidence expectation, page 36.